Digital VLSI Design — Assignment 1

4-bit Array Multiplier: RTL to GDSII

Jay Daulatkar (2023102044) Parth Tokekar (2023102041)

1 RTL design and verification

The design registers two unsigned 4-bit operands and an 8-bit product, giving 16 register bits with asynchronous active-low reset. Its array has 16 partial-product AND operations and 12 full adders in three rows. Each full adder uses continuous assignments:

$$s = a \oplus b \oplus c_{\text{in}}, \qquad c_{\text{out}} = ab + ac_{\text{in}} + bc_{\text{in}}.$$

The plus sign denotes Boolean OR. No multiplication operator is used in the RTL. Each row supplies a product bit and passes its remaining sums and carry to the next row.

1.1 Simulation

The testbench uses a 10 ns clock and holds reset low for the first two cycles. It tests all 256 input pairs, waits for two rising edges after applying each pair, and compares the output with the expected product. It also checks that reset clears the output and writes a VCD waveform.

```
PASS: 256 vectors checked, 0 errors
```

To check the testbench, we simulate a temporary copy of the full adder with the sum changed to `a ^ b`. Omitting the carry input produces the following failures:

```
MISMATCH at 1060 ns: a=3 b=3 expected=9 got=5
...
FAIL: 256 vectors checked, 95 errors
```

The original full-adder equation passes again. A separate reset fault, changing the reset value of the output from 0 to 5 in a temporary copy, produces one error at 20 ns.

1.2 Worked example

We use Jay's roll number, 2023102044. Its last digit is 4, so the assignment gives $A = 4 + 5 = 9$ and $B = 7$.

```
      1001       A = 9
    x 0111       B = 7
  --------
  00001001       9 x b[0]
  00010010       9 x b[1], shifted by 1
  00100100       9 x b[2], shifted by 2
+ 00000000       9 x b[3], shifted by 3
  --------
  00111111       P = 63
```

1.3 Waveform and latency

Figure 1 shows the VCD output from the RTL simulation. The values agree with the GTKWave trace in Figure 2. At 3040 ns the testbench applies $a = 9$, $b = 7$. The input registers capture them at 3045 ns, and the output register captures 63 at 3055 ns.

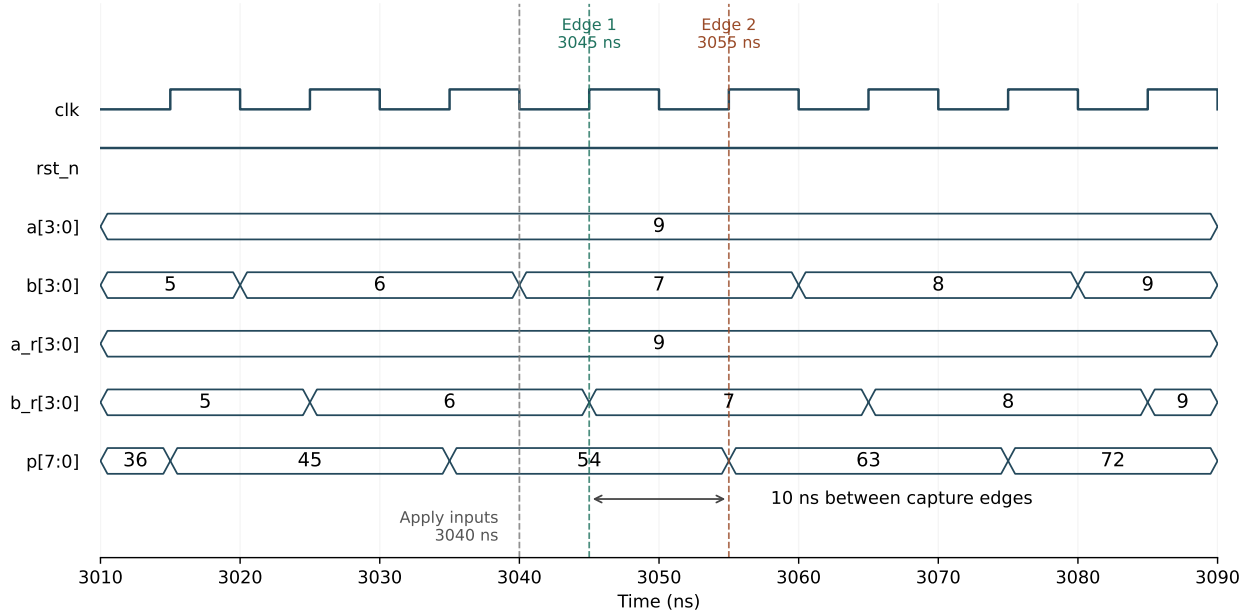


Figure 1: The two rising edges for $9 \times 7 = 63$. Edge 1 captures the operands; edge 2 captures their product. Bus values are decimal.

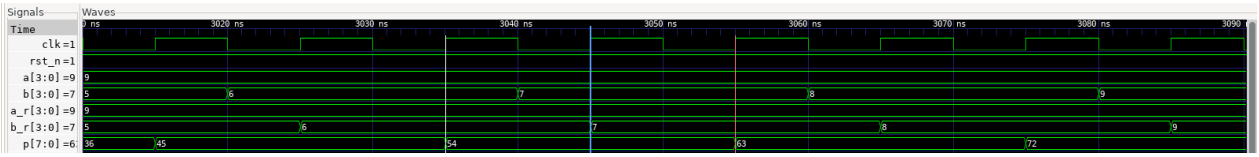


Figure 2: GTKWave view of the clock, reset, operands, registered operands and product for the same example.

Timing convention. The assignment calls this a two-cycle latency because the operands and result are captured on two successive rising edges. Those edges are 10 ns apart in this simulation. The interval from input application at the falling edge to the output update is 15 ns.

For the assignment's results table we use its stated latency expression, $2T$. This gives 5.58 ns at the 2.79 ns period found in Part C. Once the pipeline is filled, the hardware can accept a new input pair every clock cycle. The exhaustive testbench holds each pair for two cycles, so its stimulus rate is lower than the hardware's maximum throughput.

1.4 Array structure

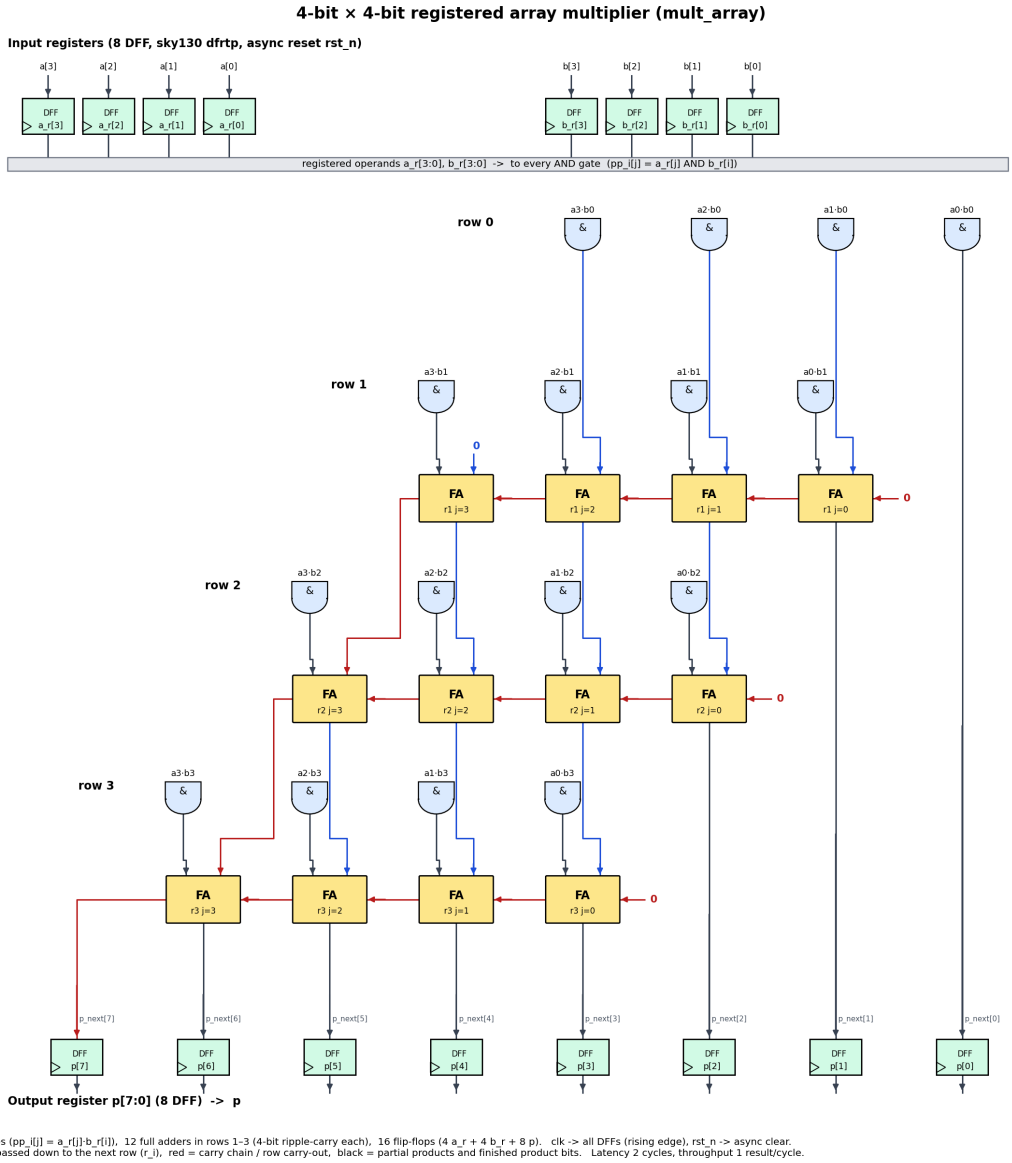


Figure 3: The implemented array: eight input-register bits, 16 AND operations, three rows of four full adders, and eight output-register bits. Blue paths pass sums between rows; red paths carry the ripple carry. The diagram's connections agree with the RTL.

2 Logic synthesis

Yosys reads and flattens the RTL, maps the flip-flops and combinational logic to SKY130 cells, removes unused wires, and writes the netlist and area report. Synthesis uses the high-density library at the typical corner: 25 °C and 1.80 V. The library identifier is `sky130_fd_sc_hd`.

Table 1: Synthesis results.

Metric	Value
Total cells	61
Flip-flops	16, type `dfrtp_1`
Total cell area	756.976 μm^2
Flip-flop area	$16 \times 25.024 = 400.384$ μm^2
Flip-flop share	52.89%
Most-used cell types	`dfrtp_1`: 16; `nand2_1`: 7; `and2_0` and `maj3_1`: 4 each, tied for third
Gate-level simulation	`PASS: 256 vectors checked, 0 errors`

The saved netlist contains the same cell counts as the synthesis report. Gate-level simulation with the SKY130 functional models passes the exhaustive testbench. It checks the mapped logic and registers; it is not a simulation with extracted routing delays.

Table 2: Mapped cells. Area is the total for all instances of each type.

Cell	Count	Area (μm^2)	Cell	Count	Area (μm^2)
`dfrtp_1`	16	400.384	`nand3_1`	2	10.010
`nand2_1`	7	26.275	`nand4_1`	2	12.512
`and2_0`	4	25.024	`nor2_1`	2	7.507
`maj3_1`	4	40.038	`xor2_1`	2	17.517
`a21oi_1`	3	15.014	`a21boi_0`	1	7.507
`xnor2_1`	3	26.275	`and3_1`	1	6.256
`xor3_1`	3	71.318	`and4_1`	1	8.758
`a22oi_1`	2	15.014	`o2111ai_1`	1	8.758
`clkinv_1`	2	7.507	`o21bai_1`	1	7.507
`lpflow_isobufsrc_1`	2	12.512	`o311ai_0`	1	8.758
			`xnor3_1`	1	22.522

3 Static timing analysis

OpenSTA reads the typical-corner library and synthesized netlist. The script creates the clock, applies 1 ns input and output delays, and reports the worst setup slack and critical path. This analysis uses an ideal clock and does not include extracted routing parasitics.

At a 10 ns period the recorded worst slack is 7.21 ns. Following the assignment's calculation,

$$T_{\min} = 10 - 7.21 = 2.79 \text{ ns}, \qquad F_{\max} = \frac{1000}{2.79} = 358.42 \text{ MHz}.$$

The saved report at 2.79 ns gives +0.003 ns slack. The period and frequency above use the assignment's rounding to 0.01 ns.

Table 3: Timing results at the typical corner.

Metric	Value
Worst slack at 10 ns	+7.21 ns
Minimum period	2.79 ns
Maximum frequency	358.42 MHz
Slack at 2.79 ns	+0.003 ns
Critical-path start	Input register `a_r[1]`, instance `_91_`
Critical-path end	Output register `p[6]`, instance `_88_`
Cells on the path	8 combinational cells; 10 instances including both flip-flops
Latency, using the assignment's $2T$	5.58 ns
Maximum throughput	358.42 million results per second

3.1 Critical path

Table 4: Ordered data path from the saved timing report. Displayed delays are rounded independently.

Pin	Cell	Increment (ns)	Arrival (ns)
`_91_/Q`	`dfrtp_1`	0.398	0.398
`_37_/Y`	`clkinv_1`	0.073	0.471
`_42_/Y`	`o311ai_0`	0.356	0.827
`_45_/Y`	`a21oi_1`	0.153	0.980
`_51_/X`	`maj3_1`	0.402	1.382
`_58_/X`	`xnor3_1`	0.370	1.752
`_61_/X`	`maj3_1`	0.368	2.119
`_63_/X`	`maj3_1`	0.354	2.474
`_64_/X`	`xor3_1`	0.178	2.652
`_88_/D`	`dfrtp_1`	0.000	2.652

The data arrives at 2.652 ns. Subtracting the 0.135 ns setup time from the 2.790 ns period gives a required time of 2.655 ns, leaving 0.003 ns slack. The 0.398 ns clock-to-Q delay is already included in the arrival time.

3.2 Relation to the adder array

The corresponding RTL route begins at `a_r[1]` and runs through the carry and sum logic of the following full adders, written as (row, column):

$$(1,0) \to (1,1) \to (2,0) \to (2,1) \to (2,2) \to (3,1) \to (3,2) \to (3,3).$$

This route passes through eight full-adder stages, matching the assignment's approximate $2N = 8$ estimate for a 4-bit array. It crosses between rows and then follows the carry chain of the last row.

The eight mapped combinational cells are not eight intact full adders. Flattening allows logic from neighbouring adders and partial-product gates to be combined. In particular, the first three cells on the mapped path form optimized logic through the early rows. The later cells produce carry or sum functions: `_51_` gives the row-2 carry, `_58_` an inverted row-2 sum, `_61_` and `_63_` inverted row-3 carries, and `_64_` the final sum for product bit 6. Gate count and full-adder stage count happen to be equal on this path, but they measure different things.

4 Physical design

LibreLane takes the RTL through synthesis, floorplanning, placement, clock-tree synthesis, routing, extraction and signoff. The design uses a fixed 150 μm $\times$ 150 μm die, with `clk` as its clock input.

The initial period follows the assignment:

$$1.2T_{\text{min}} = 1.2 \times 2.79 = 3.348 \text{ ns},$$

rounded upward to the next 0.5 ns step, giving 3.5 ns. The README records ten attempts, with the period increased by 0.5 ns after each setup failure. The final period is 8.0 ns.

Table 5: Recorded place-and-route attempts. The first nine rows are the history retained in the README; the final row agrees with the saved metrics.

Run	Period (ns)	Worst slack (ns)	Nominal TT slack (ns)	Result
1	3.5	−2.188	+0.509	Setup failure
2	4.0	−1.688	+1.009	Setup failure
3	4.5	−1.188	+1.509	Setup failure
4	5.0	−0.686	+2.009	Setup failure
5	5.5	−0.208	+2.509	Setup failure
6	6.0	−0.154	+2.877	Setup failure
7	6.5	−0.220	+3.119	Setup failure
8	7.0	−0.134	+3.520	Setup failure
9	7.5	−0.104	+3.737	Setup failure
10	8.0	+0.396	+4.237	Passed

The README records zero Magic, KLayout and LVS errors for every attempt. Only the final run's metrics and layout are retained in the submitted folder, so the earlier attempts cannot be independently checked from those files.

4.1 Final results

Table 6: Physical-design results from the final saved metrics.

Metric	Value
Clock period	8.0 ns
Recorded attempts	10
Core area	17,759.5 μm^2
Die area	22,500 μm^2
Cells after Part B synthesis	61
Instances after place and route	1,883
Added cell types	Taps, decoupling capacitors, fillers, timing-repair buffers and clock buffers
Routed wirelength	2,499 μm
Worst setup slack	+0.396 ns
Worst hold slack	+0.136 ns
Post-route frequency estimate	131.51 MHz
Part C frequency estimate	358.42 MHz
DRC errors	0 in Magic, KLayout and the detailed router
LVS errors	0

Using the assignment's formula and the saved worst-corner slack,

$$F_{\text{max,route}} = \frac{1000}{8.0 - 0.396259925} = 131.51 \text{ MHz}.$$

This is about 36.7% of the Part C estimate. Part C uses typical cells, an ideal clock and no routed parasitics; the final worst case combines slow-corner cells, maximum parasitics and a real clock network. At the nominal typical corner, the saved slack of +4.23749 ns gives 265.78 MHz. This value still includes the routed implementation and its timing constraints.

4.2 Instance counts

The GDS contains 1,883 top-level instances: 245 taps, 1,298 decoupling cells, 254 fillers, 21 timing-repair buffers, three clock buffers and 62 other logic/register cells. The last group contains 16 flip-flops, 42 other combinational cells and four inverters. LibreLane performs its own synthesis, so its logic count need not equal Part B's 61 cells.

The taps and decap/fill cells account for 1,797 instances. The remaining 86 are logic, registers and buffers. Their area is approximately 958.4 μm^2, or 5.40% of the core. Including taps gives 1,264.96 μm^2, or 7.12%; this larger figure is not the logic utilization alone.

4.3 Remaining electrical warning

The metrics report 11 maximum-slew violations in the slow corner, with no maximum-fanout or maximum-capacitance violations. The design meets the assignment's stated final criteria: nonnegative setup slack, zero DRC errors and zero LVS errors. The slew warnings still need attention for complete electrical closure. The saved aggregate metrics do not identify individual pins, so they do not by themselves establish the affected net or its recovery/removal margins.

4.4 Layout

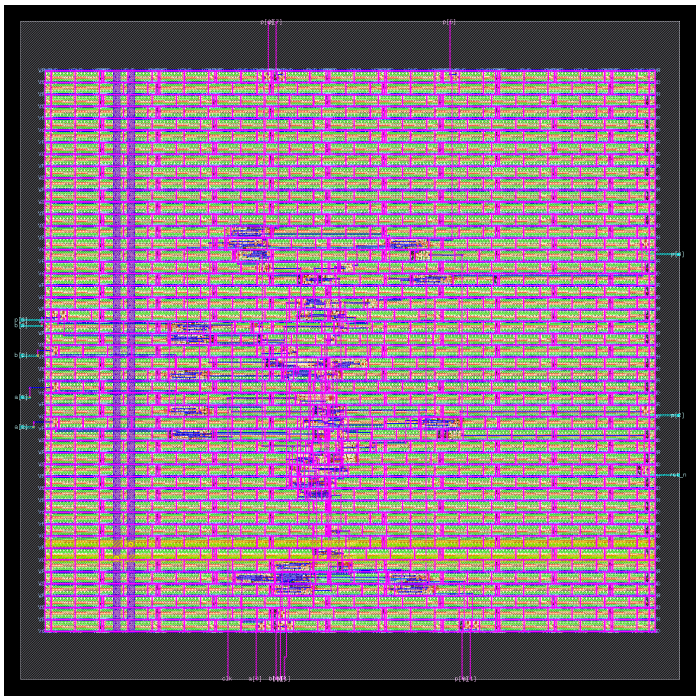


Figure 4: Whole-die KLayout view. The 150 μm square die contains the power grid, routed logic and rows filled with tap, decoupling and filler cells.

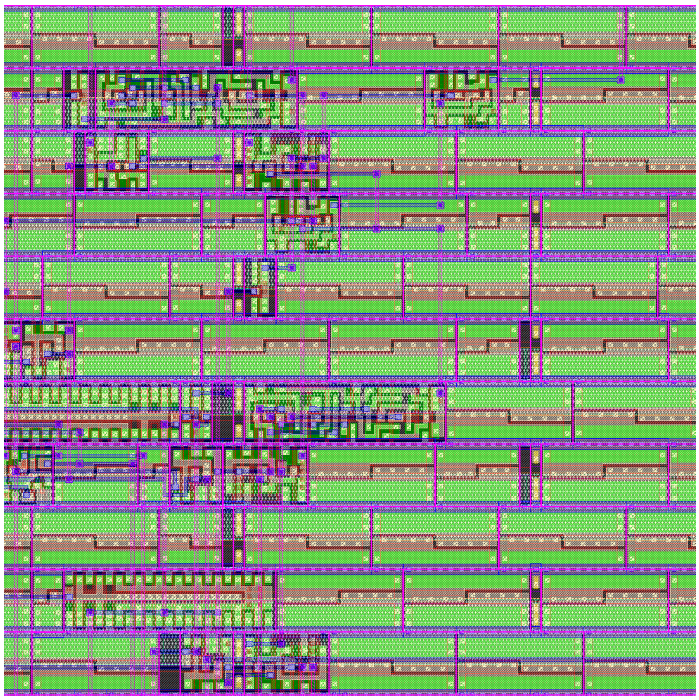


Figure 5: A closer view of standard-cell rows, routed metal and vias in KLayout.

5 Discussion

5.1 Why synthesis produces 45 logic cells

Synthesis gives $61 - 16 = 45$ non-register cells. The 16 AND operations and 12 full adders are RTL functions, not library-cell instances. Flattening lets Yosys and ABC combine logic across adder boundaries. Sum and carry functions may require separate cells, while constant inputs and shared logic allow others to be simplified or merged. The netlist has four majority cells, four three-input XOR/XNOR cells and 37 other combinational cells, rather than one cell for each RTL operation.

5.2 Critical path and the $2N$ estimate

The critical path runs from `a_r[1]` to `p[6]` through eight combinational cells, or ten instances including both endpoint flip-flops. Its RTL route passes through eight full-adder stages, matching the approximate $2N = 8$ estimate. A full-adder stage is not equivalent to one mapped gate: synthesis combines logic, and the 2.652 ns arrival time also includes 0.398 ns of launch clock-to-Q delay.

5.3 Flip-flop area

The 16 flip-flops occupy 400.384 μm^2 out of 756.976 μm^2, giving 52.89%. Each resettable flip-flop occupies 25.024 μm^2, about 3.2 times the average combinational cell area of 7.92 μm^2. The register overhead is large for this small array. Register count grows as $4N$, while the arithmetic grows quadratically, so wider arrays generally devote a smaller fraction of their area to registers.

5.4 Frequency after routing and added cells

The routed estimate falls from 358.42 to 131.51 MHz because the physical timing model includes wire RC and a real clock tree, and its worst case uses slow-corner cells. The nominal typical-corner routed estimate is 265.78 MHz. Taps connect wells and substrate to the supplies; decoupling cells provide local supply capacitance; fillers maintain row continuity. The 21 timing-repair buffers adjust signal drive and delay, and three clock buffers distribute the clock. Physical timing constraints and logic mapping also differ from Part C.

5.5 DRC and LVS

DRC checks geometry against process rules such as minimum width, spacing and enclosure. LVS extracts circuit connectivity and compares it with the reference netlist. Both are needed: correct connectivity does not guarantee manufacturable geometry, and geometrically legal shapes can still be connected incorrectly. The saved metrics report zero DRC and LVS errors. These checks do not prove that every electrical limit is met; the slew warnings are a separate issue.

5.6 Scaling to 8 bits

The full-adder count becomes $8(8-1) = 56$, versus 12 for four bits, an increase of 4.67 times. AND operations increase from 16 to 64 and register bits from 16 to 32. The approximate critical path doubles from eight to 16 full-adder stages. Doubling the measured logic delay gives $T \approx 0.398 + 2(2.652 - 0.398) + 0.135 = 5.041$ ns, or about 198 MHz. This is an estimate: actual loading, mapping and routing can change it. The assignment's two-cycle latency convention stays the same.

Verification notes

The repeat RTL simulation passes all 256 inputs; full-adder and reset-value fault injection produce 95 and one errors respectively. An independent bit-level check of all 256 VCD vectors confirms the row sums and carries. The saved netlist agrees with all synthesis cell counts. Earlier gate-level simulation with official SKY130 models passed all 256 inputs.

The GDS has 33 structures, 1,883 top-level instances and 40 layer/datatype pairs, with no unresolved references. These consistency checks do not replace signoff: the DRC, LVS and timing results in this report come from saved outputs; the full flow was not rerun locally because the EDA image and PDK were unavailable.

The recorded tools are Icarus Verilog 14.0, Yosys 0.70, OpenSTA 3.1.0, LibreLane 3.1.0.dev3 and KLayout 0.30.12 in IIC-OSIC-TOOLS 2026.10. The repeat RTL checks used Icarus Verilog 11.0. Commands and the image digest are in the README.

6 GitHub repository

The source files, commands and submitted design outputs are available at:

https://github.com/jaydaulatkar/DVD_A1

7 Member contributions

Jay Daulatkar (2023102044). Led the core design and implementation: designed the full adder and registered 4-bit array multiplier RTL, implemented the Yosys synthesis and OpenSTA timing-analysis scripts, and configured the LibreLane RTL-to-GDSII flow on SKY130. Also helped create the report, reviewed Parth's draft, provided feedback, and revised the report for clarity and technical accuracy.

Parth Tokekar (2023102041). Developed the exhaustive self-checking testbench, performed RTL and gate-level simulations, checked reset behavior and latency, and verified that fault injection was detected. Ran the synthesis, timing-analysis and physical-design flows to generate and check the results and final GDSII layout. Reviewed assignment requirements, provided feedback, and helped diagnose and fix bugs. Prepared and organized the report, including the results tables, waveform and layout figures, explanations and answers to the assignment questions.